"Build once. Compose everywhere. Ship reusable UI."

MODULE 33

React Component Architecture

Build reusable, scalable, and maintainable user interfaces with React components, JSX, props, and composition.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

MODULE 33 OVERVIEW

Modern enterprise applications need fast, interactive, maintainable user interfaces -- React's component model is the industry-standard way to build them.

Enterprise systems like the Customer Management Platform CRM need a frontend that's as disciplined as the backend: typed, composable, accessible, and built from small, tested, reusable pieces.

DURATION & LAB

Module Duration

1 Hour Theory · 2.5 Hours Lab · 3.5 Hours Total

Theory Focus

React fundamentals, JSX, props, composition, and modular UI design.

Business Scenario

Build the Customer Management Platform CRM dashboard component shell.

WHY A COMPONENT-BASED FRONTEND MATTERS

Without Component-Based UI	With React Components
Monolithic, tangled UI code -- hard to maintain or extend.	Modular, composable UI -- small, reusable components.
Duplicated markup and logic copy-pasted across pages.	Write once, reuse everywhere via props and composition.
Manual DOM updates -- slow, error-prone, hard to reason about.	Declarative rendering -- the Virtual DOM updates only what changed.
Inconsistent UI and accessibility across teams.	Consistent, accessible, well-tested components (StatusBadge, CustomerCard).
Slow onboarding -- new developers learn ad-hoc conventions.	Faster onboarding -- clear component contracts and typed props.

KEY TAKEAWAY React's component architecture turns a tangled frontend into a modular, typed, accessible system built from small, reusable, well-tested pieces -- exactly what Lab 33's CRM dashboard requires.

WHY REACT IS POPULAR

React has become the go-to library for building user interfaces -- here's why developers and enterprises love it.

High Performance

Uses a Virtual DOM to update only what changed, keeping apps fast.

Component Based

Build reusable UI components and compose complex interfaces easily.

Declarative

Makes your code predictable and easier to debug and maintain.

Large Ecosystem

Huge community support and a rich ecosystem of libraries and tools.

Cross-Platform

React Native builds iOS and Android apps from shared logic.

Scalable for Any Size

Scales from small projects to large enterprise systems with ease.

Backed by Meta

Maintained by Meta, with continuous improvements and stability.

Developer Friendly

Easy to learn, with great docs and powerful DevTools.

INDUSTRY ADOPTION

Facebook **Instagram** **Netflix** **Airbnb** **Tesla** **Thousands of enterprises**

KEY TAKEAWAY React empowers developers to build modern, interactive, and maintainable user interfaces faster -- while delivering exceptional user experiences.

Why React is Popular

React is a powerful JavaScript library for building user interfaces, trusted by developers and companies worldwide.

1

Component-Based Architecture

Build reusable UI components that are easy to manage, maintain, and scale.

2

High Performance

Virtual DOM updates only the changed parts of the UI, delivering a fast user experience.

3

Declarative Syntax

Write simpler and more predictable code, making development faster and easier.

4

Rich Ecosystem

Large collection of libraries, tools, and extensions to accelerate development.

5

Strong Community

Backed by a large, active community that continuously shares knowledge and support.

6

Backed by Meta

Maintained by Meta and used in thousands of real-world applications at scale.

7

Cross-Platform Capabilities

Use React Native to build high-quality mobile apps with the same skills.

8

Future-Ready

Regular updates, new features, and strong roadmap ensure long-term reliability.

Used by Top Companies:

Facebook, Instagram, WhatsApp Web, Netflix, Airbnb, Tesla, and many more trust React to deliver exceptional user experiences.

MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to develop well-structured, reusable, and dynamic React components.

- **Understand React Fundamentals —** explain what React is, its architecture, and the basic structure of a React application.
- **Build Components —** create functional components and understand the component lifecycle at a high level.
- **Work with JSX —** use JSX syntax to describe UI and render dynamic content in a React application.
- **Use Props Effectively —** pass data between components using props and make components reusable.
- **Compose Components —** combine components to build complex, modular, and maintainable user interfaces.
- **Design Modular UIs —** apply best practices to design scalable, maintainable, and enterprise-ready UIs.
- **Apply Best Practices —** follow React best practices for component structure, naming, and code organization.
- **Build in the Lab —** develop, test, and enhance a React application by building reusable components.

KEY TAKEAWAY You will be able to develop well-structured, reusable, and dynamic React components to build modern, user-friendly, and enterprise-grade web applications.

WHAT IS REACT?

React is a JavaScript library for building user interfaces, especially single-page applications where you need a fast, interactive experience.

KEY HIGHLIGHTS

- **Component Based —** build encapsulated, reusable UI components.
- **Declarative —** describe how the UI should look for a given state; React handles the updates.
- **Efficient —** uses a Virtual DOM to update only the parts of the UI that actually changed.
- **Flexible & Scalable —** an ecosystem of libraries and tools, from small widgets to large enterprise applications.

A SIMPLE EXAMPLE — Welcome.jsx

```
import React from "react";

function Welcome(props) {
  return <h1>Hello, {props.name}! 👋</h1>;
}

export default Welcome;
```

React helps developers build reusable UI components and efficiently update and render the right components when data changes.

Reusable Components **High Performance** **Scalable Ecosystem**

Trusted by Industry Leaders

HOW REACT WORKS

1. **Write JSX**
2. **Virtual DOM**
3. **Diffing**
4. **Update Real DOM**

KEY TAKEAWAY React is a powerful library to build fast, maintainable, and scalable user interfaces using reusable components and a highly efficient rendering model.

What is React?

React is a JavaScript library for building fast, scalable, and interactive user interfaces using a component-based architecture.

React

A JavaScript Library for Building User Interfaces

Declarative

Describe what the UI should look like for every state, and React handles the updates.

Component-Based

Build reusable UI components and compose them together to create complex user interfaces.

Efficient

React updates only the changed parts of the UI using a Virtual DOM for better performance.

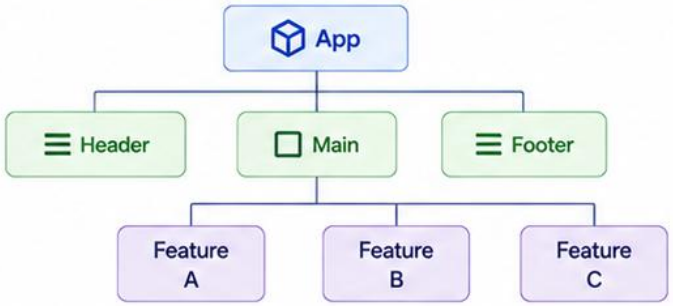
Flexible

Easily integrate with other libraries and tools to build single-page or complex web applications.

Popular & Trusted

Maintained by Meta and used by millions of developers and companies worldwide.

Component-Based Architecture



How React Works

State Changes
Data or state changes in the application.

→

React Re-renders
React creates a new Virtual DOM in memory.

→

Diffing
React compares the new Virtual DOM with the previous one.

→

Updates UI
React updates only the parts of the UI that changed.

Key Takeaway

React helps you build modern, fast, and maintainable user interfaces by breaking the UI into reusable components and efficiently updating only what's necessary.

REACT ARCHITECTURE OVERVIEW

React follows a component-based architecture with a unidirectional data flow to build fast, scalable, maintainable user interfaces.

CORE PRINCIPLES

Component-Based | **Unidirectional Data Flow** | **Virtual DOM** | **Declarative**

KEY BUILDING BLOCKS

Block	What It Does
Components	Building blocks of a React app; functional or class.
JSX	A syntax extension for writing HTML-like code in JS.
Props	Read-only inputs passed from parent to child.
State	Local data managed within a component, can change.
Hooks	Functions that let you use state and other features.
Context	Shares data across the component tree without prop drilling.

HOW DATA FLOWS IN REACT

1. **User Interacts**
2. **Updates State**
3. **React Re-renders**
4. **DOM Updates**

THE THREE ARCHITECTURE LAYERS

- **UI Layer —** your components (App, Header, CustomerList, ...).
- **React Core —** lifecycle, reconciliation (diffing), the Virtual DOM.
- **Data Layer —** state, data, and APIs that feed the UI layer.

Data flows down via props; user events flow up via callbacks -- this predictable, one-directional loop is what makes large React apps easy to reason about.

KEY TAKEAWAY React is built around components and unidirectional data flow: props flow down, events flow up via callbacks -- an architecture that enables scalable, maintainable, high-performance apps.

REACT APPLICATION STRUCTURE

A well-structured React application makes your code easier to understand, maintain, and scale.

WHAT EACH FOLDER/FILE DOES

Folder/File	Purpose
public/	Static files served as-is; index.html is the main HTML file.
src/	Contains all the source code of the application.
assets/	Images, icons, fonts, and other static assets.
components/	Reusable UI components used across pages (Header, Button).
pages/	Page-level components for different routes/views.
App.jsx	The root component that composes the UI and defines routes.
index.jsx	Entry point; renders <App /> into the DOM.
package.json	Project metadata, dependencies, and scripts.

APP.JSX (EXAMPLE)

```
import Header from "./components/Header";
import Footer from "./components/Footer";
import Home from "./pages/Home";
function App() {
  return (
    <>
      <Header />
      <Home />
      <Footer />
    </>
  );
}
export default App;
```

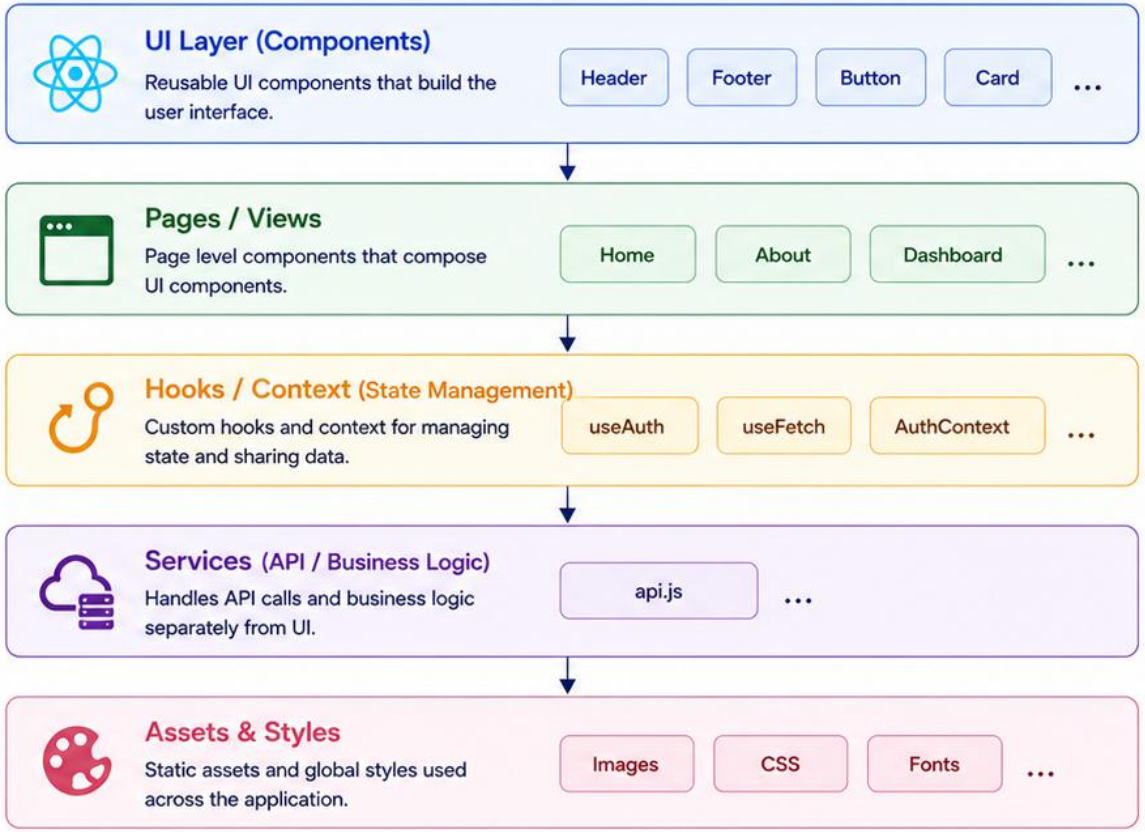
BEST PRACTICES

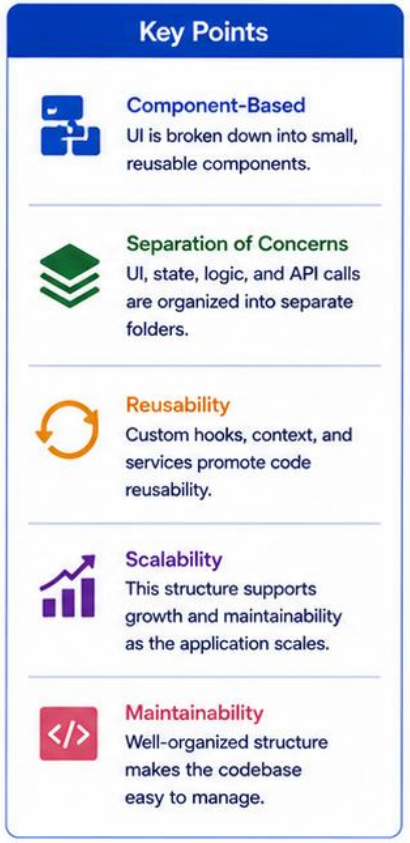
- Keep components small
- Group files by feature
- Meaningful names
- Reuse components
- Consistent structure

KEY TAKEAWAY Components are organized by feature and reusability; index bootstraps the app; App.jsx composes the UI -- a structure that stays scalable and easy to maintain as the app grows.

React Application Structure

A typical React application is organized into reusable and maintainable layers.





FUNCTIONAL COMPONENTS

Functional components are JavaScript functions that return JSX -- simpler, easier to read, and the preferred way to write components in modern React.

KEY CHARACTERISTICS

- **JavaScript Function —** created using a plain JavaScript function.
- **Returns JSX —** the function returns JSX that describes what to render.
- **No this Keyword —** they don't have their own instance or this context.
- **Reusable —** accept props as input and can be reused anywhere.
- **Lightweight —** easier to test, maintain, and reason about.

EXAMPLE: FUNCTIONAL COMPONENT — Welcome.jsx

```
import React from "react";

// Functional component
function Welcome(props) {
  return (
    <div className="welcome">
      <h2>Hello, {props.name}! 👋</h2>
      <p>Welcome to React components.</p>
    </div>
  );
}
export default Welcome;
```

COMPARISON: CLASS VS FUNCTIONAL

Aspect	Class	Functional
Syntax	ES6 class	Plain function
State	this.state	useState Hook
Lifecycle	Many methods	useEffect Hook
Code Size	More verbose	More concise
Recommended	Legacy/complex	Preferred in modern React

WHY USE FUNCTIONAL COMPONENTS?

Cleaner syntax **Easier to test** **Work with Hooks**

Better performance **Community recommended**

KEY TAKEAWAY Functional components with Hooks are the modern, recommended way to write React -- cleaner syntax, easier testing, and full feature parity with class components.

TRY IT YOURSELF — EXERCISE 1: FILL COMPONENT TODOS (STARTER)

Reinforces: basic functional components, typed props, and JSX — fill in every blank before moving on.

FILL IN EVERY ____ (notes/lab33-todos.md)

CustomerCard.tsx (STARTER — not a solution)

```
type CustomerCardProps = {
  customerId: string;
  name: string;
  status: _____;
  onSelect: (id: string) => void;
};

export function CustomerCard(
  { customerId, name, status, onSelect }: CustomerCardProps
) {
  return (
    <article aria-label={_____}>
      <h3>{_____}</h3>
      <StatusBadge status={status} />
      <button type="button" onClick={() => onSelect(_____)}>
        View
      </button>
    </article>
  );
}
```

#	What To Do
1	Paste the stub above into notes/lab33-todos.md exactly as shown.
2	Fill blanks with: 'ACTIVE' | 'SUSPENDED', `${name} (${customerId})`, name, customerId.
3	Add a filled sample usage: <CustomerCard customerId="CUS-1001" name="Amina Khan" status="ACTIVE" onSelect={...} />.
4	Write an a11y note: the button has visible text; avoid icon-only buttons without aria-label.

This starter is intentionally simpler than Lab 33's real CustomerCard (no onEdit, no mailto link) -- master typed props and JSX here first.

KEY TAKEAWAY TRY IT NOW — This is a STARTER, not a solution: fill every blank in exercises/exercise-01-fill-react-component-todos.md before moving on.

COMPONENT LIFECYCLE OVERVIEW

The component lifecycle is the series of phases a component goes through, from being created to being removed from the DOM.

LIFECYCLE PHASES

- **1. Mounting —** the component is being created and inserted into the DOM.
- **2. Updating —** the component updates due to changes in props or state.
- **3. Unmounting —** the component is being removed from the DOM.

CLASS LIFECYCLE METHODS

Method	When It Runs
constructor()	Component initialized.
render()	Every time it renders.
componentDidMount()	After added to the DOM.
componentDidUpdate()	After props/state change.
componentWillUnmount()	Just before removal.

FUNCTIONAL EQUIVALENT: useEffect

```
import { useEffect } from "react";

function Example({ data }) {
  useEffect(() => {
    console.log("mounted");
    return () => {
      console.log("cleanup");
    };
  }, [data]); // re-runs when data changes

  return <div>Data: {data}</div>;
}
```

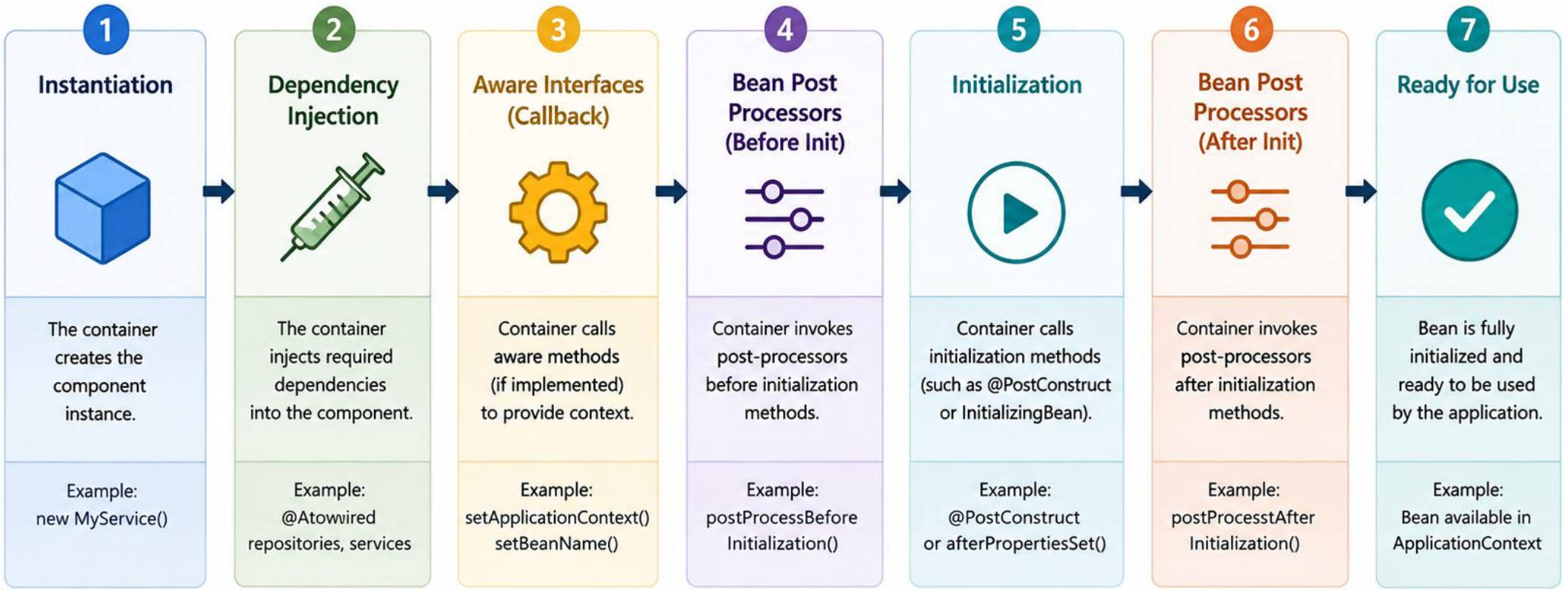
Understanding these phases helps manage resources, side effects, and performance -- and prevents memory leaks.

BEST PRACTICES

- **Side effects in useEffect**
- **Clean up in the return function**
- **Avoid unnecessary updates**
- **Use Hooks for lifecycle-like behavior**

KEY TAKEAWAY Functional components use the useEffect Hook to handle mounting, updating, and cleanup -- keep side effects in useEffect and clean up resources in its return function.

Component Lifecycle Overview



Container Actions in Summary

COMPONENT COMPOSITION

Component composition is the process of combining simple, reusable components to build complex user interfaces.

WHY COMPONENT COMPOSITION?

- **Reusability —** build once, use everywhere.
- **Maintainability —** smaller components are easier to maintain and update.
- **Scalability —** easily combine and extend components as your app grows.
- **Team Collaboration —** small, focused components are easier to develop in parallel.

COMPONENT HIERARCHY

In the lab, this maps directly to App → CustomerList → CustomerCard → StatusBadge -- each level composes the level below it, and data (props) flows straight down that same chain.

IN THE CRM DASHBOARD — App.tsx

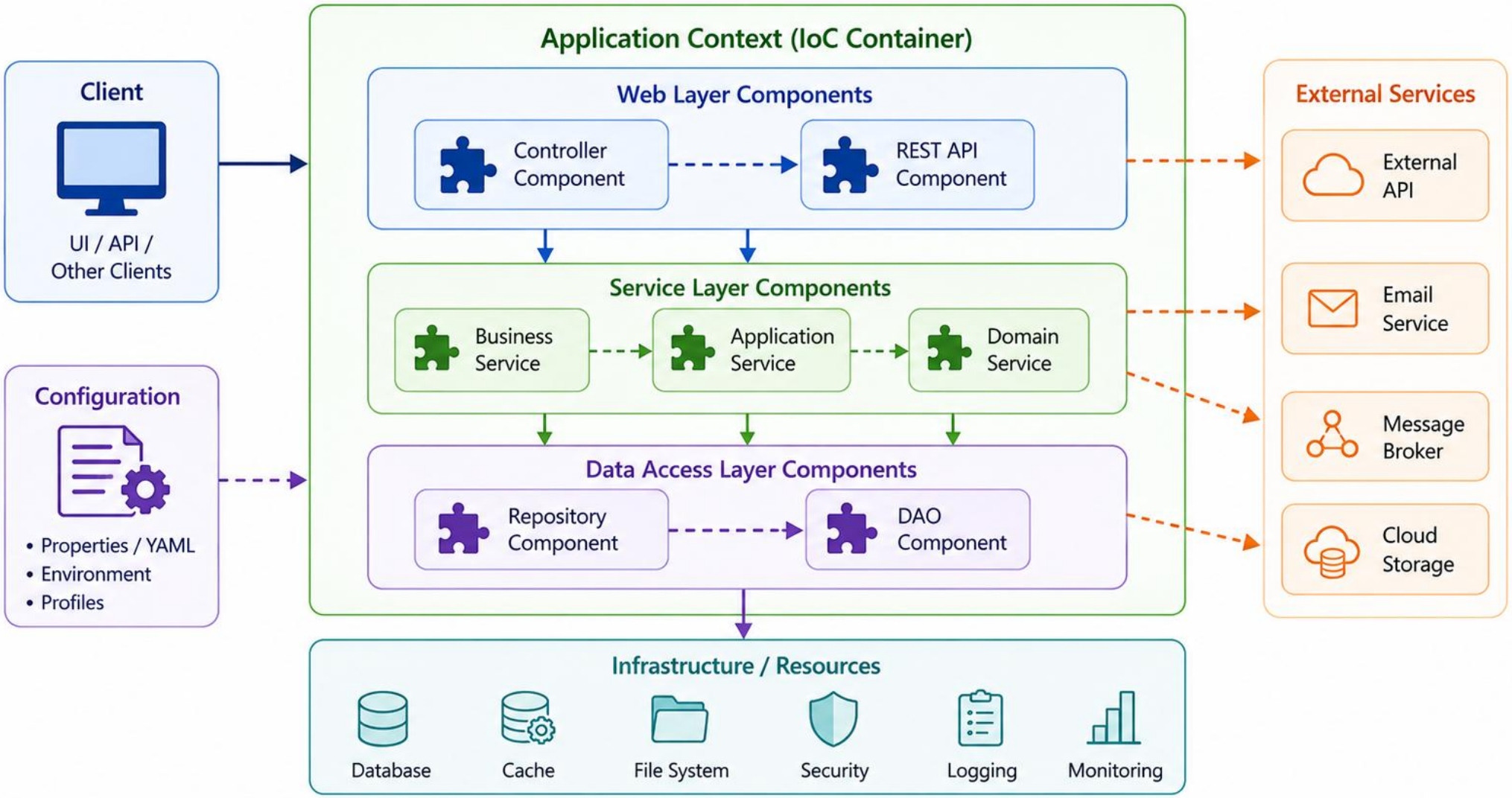
```
import { CustomerList } from "./components/CustomerList";

function App() {
  return (
    <AppLayout>
      <CustomerToolbar onAdd={handleAdd} />
      <CustomerList
        customers={seedCustomers}
        onEdit={handleEdit}
      />
    </AppLayout>
  );
}
```

BEST PRACTICES

- Single responsibility
- Compose via props & children
- Composition over inheritance
- Favor reusability

KEY TAKEAWAY Compose small, focused components together to build powerful, flexible, maintainable UIs -- exactly the CustomerList → CustomerCard → StatusBadge composition you'll build in the lab.

Component Composition



TRY IT YOURSELF — EXERCISE 2: COMPONENT INVENTORY

Reinforces: breaking a UI into components with single responsibilities — an analysis exercise.

EXAMPLE STARTING POINT (≥ 5 COMPONENTS)

Component	Responsibility (≤ 6 words)
App	Root shell; composes the dashboard
CustomerList	Renders one card per customer
CustomerCard	Displays a single customer's info
StatusBadge	Shows a customer's status text
PageHeader	Displays page title and actions

STEPS (IN notes/lab33-components.md)

#	What To Do
1	Screen: imagine a Customer list showing Amina and Ravi with status badges.
2	Inventory: list at least 5 components (use the starting point at left, or your own).
3	One Responsibility: for each component, write a responsibility in 6 words or fewer.
4	Notes: save the finished inventory as notes/lab33-components.md.

Compare your inventory to Lab 33's real suggested files -- StatusBadge, CustomerCard, CustomerList, CustomerForm, AppLayout, CustomerToolbar -- most of what you listed already has a real counterpart there.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 2 before continuing: exercises/exercise-02-component-inventory.md (workspace: examples/module-33-exercises).

REUSABLE COMPONENTS

Reusable components are generic, self-contained UI pieces that can be used in multiple places across an application.

GOOD REUSABLE COMPONENT CHARACTERISTICS

- **Single Responsibility —** does one thing well.
- **Configurable via Props —** accepts data and behavior through props.
- **Flexible —** works in different contexts and layouts.
- **Independent —** does not rely on external state or side effects.
- **Well Documented —** easy to understand and use.

EXAMPLE: BUTTON.JSX

```
function Button({ label, type = "primary", onClick }) {
  const cls = type === "primary" ? "btn-primary" : "btn-secondary";
  return (
    <button className={`btn ${cls}`} onClick={onClick}>
      {label}
    </button>
  );
}
export default Button;
```

HOW TO BUILD REUSABLE COMPONENTS

1. Identify
2. Extract
3. Configure
4. Reuse
5. Refactor

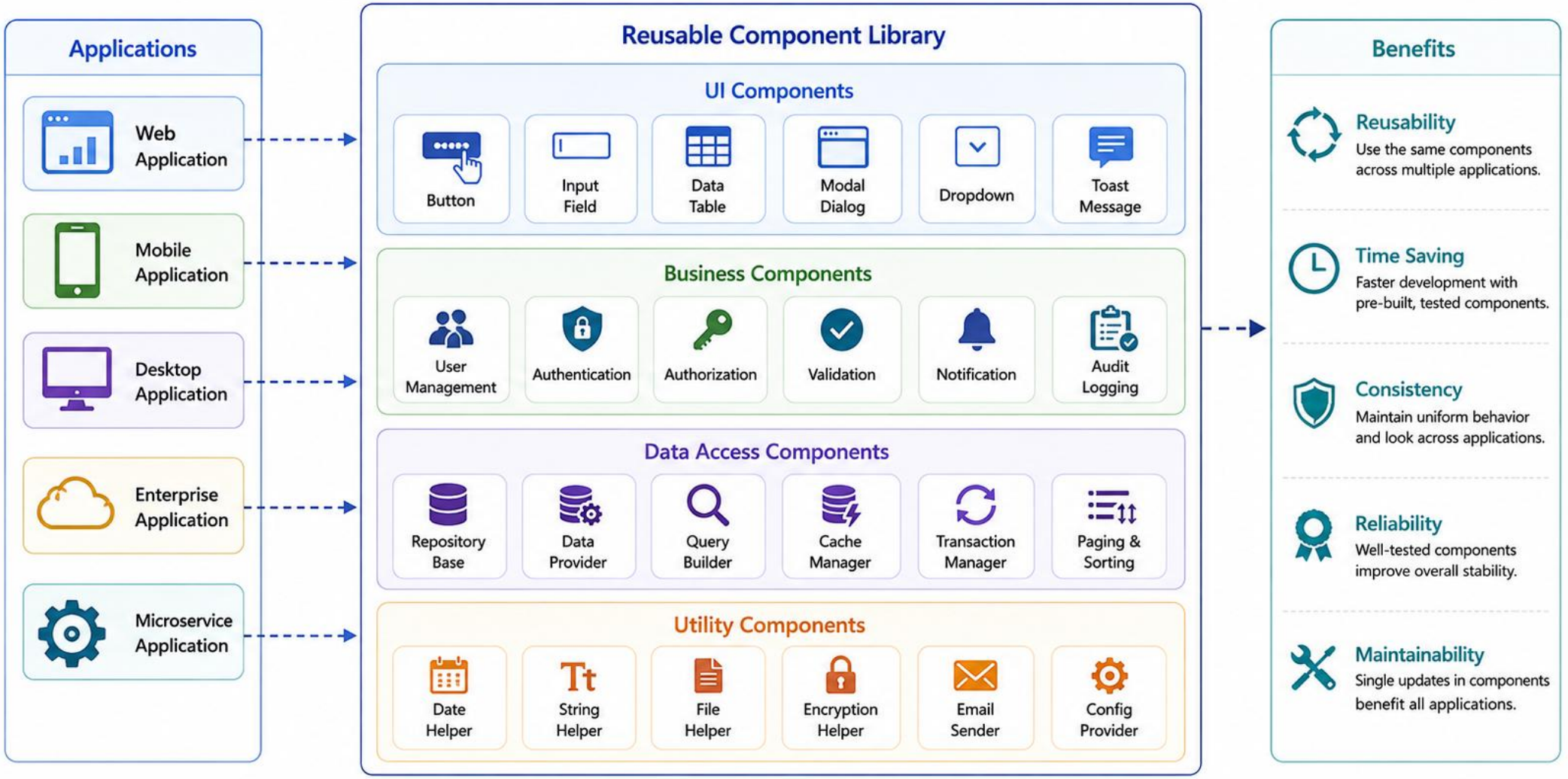
BEST PRACTICES

- Name clearly
- Sensible default props
- Consistent styling
- Test in isolation
- Organize in a dedicated folder

KEY TAKEAWAY Reusable components help you build consistent, maintainable, and scalable React applications by maximizing code reuse and minimizing duplication.

Reusable Components

Build once, use many times across different applications.



WHAT IS JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript that allows you to write HTML-like code inside JavaScript.

WHY JSX?

Readable **Expressive** **Developer Friendly** **Powerful**

KEY POINTS

- JSX looks like HTML, but it is JavaScript.
- It allows embedding expressions using { }.
- Browsers do not understand JSX directly.
- Babel (or a similar tool) transpiles JSX into JavaScript before execution.
- React uses that output to create and update the DOM efficiently.

JSX is not a requirement of React, but it is the most common, and most readable, approach.

JSX EXAMPLE

```
const name = "Amina";
const element = (
  <div className="card">
    <h1>Hello, {name}!</h1>
  </div>
);
```

TRANSPILED TO JAVASCRIPT

```
const element = React.createElement(
  "div", { className: "card" },
  React.createElement("h1", null, "Hello, ", name, "!")
);
```

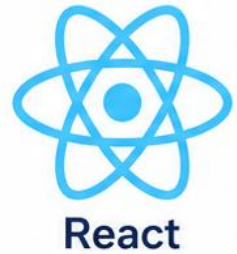
JSX (you write) **Babel (transpiles)**

JavaScript (browser runs)

KEY TAKEAWAY JSX brings HTML-like syntax into JavaScript, making React components more intuitive, readable, and maintainable. It enhances productivity and accelerates UI development.

What is JSX?

JSX (JavaScript XML) is a syntax extension for JavaScript.
It lets you write HTML-like code inside JavaScript and is compiled to React.createElement() calls.



Looks like HTML
Easy to read and write

JavaScript Expressions
Use {} to embed any JavaScript expression

Component-Based
Compose UI using reusable components

Safe & Validated
JSX is validated at compile time and prevents many errors

JSX

```jsx
function Hello() {
  const name = "React Developer";
  return (
    <div className="greeting">
      <h1>Hello, {name}!</h1>
      <p>Welcome to React with JSX.</p>
    </div>
  );
}
```

Compiles to

Compiled JavaScript (React.createElement)

```jsx
function Hello() {
  const name = "React Developer";
  return React.createElement(
    "div", { className: "greeting" },
    React.createElement("h1", null, "Hello, ", name, "!"),
    React.createElement("p", null, "Welcome to React with JSX.")
  );
}
```

Benefits of JSX

Improved Readability
HTML-like syntax makes UI structure easy to visualize.

Faster Development
Write less code and build UI quickly.

Error Reduction
Compile-time checks catch common mistakes.

Better Maintainability
Clean, component-based structure is easy to maintain.

Powerful & Flexible
Combine JavaScript logic directly in your UI.

JSX SYNTAX RULES (1 OF 2)

JSX follows specific syntax rules that ensure your code is valid and can be transpiled to regular JavaScript.

Rule	Correct	Incorrect
1. Return a single root element	return (Hi Bye);	return (Hi Bye);
2. Close all tags		<img src="logo.png" alt="Logo">
3. Use className instead of class		
4. Use htmlFor instead of for	<label htmlFor="name">Name</label>	<label for="name">Name</label>
5. Use curly braces for expressions	Hello, {userName}!	Hello, userName!
6. Use camelCase for attributes	<input readOnly tabIndex={0} />	<input readonly tabindex="0" />

WHY THESE RULES MATTER

JSX compiles to JavaScript function calls -- a function can only return one value, tags map to real DOM elements that must be well-formed, and JSX attributes are JavaScript property names, which is why className/htmlFor/camelCase replace their HTML equivalents.

KEY TAKEAWAY Following JSX syntax rules ensures your components work correctly and your code stays clean, readable, and error-free.

</> JSX Syntax Rules

JSX looks like HTML, but follows JavaScript rules.

1 Single Parent Element

JSX must return one root element.

```
return (
  <div>
    <h1>Hello</h1>
    <p>Welcome</p>
  </div>
)
```

```
return (
  <h1>Hello</h1>
  <p>Welcome</p>
)
```

✖ Adjacent elements not allowed.

2 Closing Tags

All JSX elements must have a closing tag.

```
<button>Click</button>
<input type="text" />
<img src="logo.png" />
```

```
<button>Click
<input type="text">
<img src="logo.png">
```

✖ Missing closing tags.

3 camelCase for Attributes

Use camelCase for all HTML attributes.

```
<div className= "box"
     htmlFor="inputId"
     tabIndex={0}>
```

```
<div class= "box"
     for="inputId"
     tabindex="0">
```

✖ Use camelCase, not lowercase.

4 Expressions in Curly Braces

Use { } to embed JavaScript expressions.

```
<h1>Hello, {name}!</h1>
<p>2 + 2 = {2 + 2}</p>
<img src={imageUrl} />
```

```
<h1>Hello, name!</h1>
<p>2 + 2 = 4</p>
<img src="imageUrl" />
```

✖ Use { } for dynamic values.

5 JavaScript Expressions Only

Only expressions, not statements or declarations.

```
<div>{user.name}</div>
{isLoggedIn ? <Home /> : <Login />}
```

```
<div>{if (user) { return user }}</div>
<div>{const x = 10}</div>
```

✖ No statements or declarations.

6 Escape HTML Characters

Use { } for dynamic values to auto-escape HTML.

```
<p>{'<script>alert("XSS")</script>"}</p>
<p>{userInput}</p>
```

```
<p><script>alert("XSS")</script></p>
<p dangerouslySetInnerHTML={{
  __html: userInput
}} />
```

✖ Prevent XSS by escaping content.

7 Self-Closing Elements

Void elements must be self-closed.

```
<br />
<hr />
<input type="text" />
<img src="image.png" />
```

```
<br>
<hr>
<input type="text">
<img src="image.png">
```

✖ Add / at the end.

8 Component Names Must Start with Capital

Custom components must start with an uppercase letter.

```
<MyComponent />
<UserCard />
```

```
<myComponent />
<userCard />
```

✖ Lowercase treated as HTML tags.

9 Comments in JSX

Use { /* ... */ } for comments inside JSX.

```
<div>
  { /* This is a comment */ }
  <p>Hello</p>
</div>
```

```
<div>
  <!-- This is a comment -->
  <p>Hello</p>
</div>
```

✖ HTML comments not allowed.

10 Boolean Attributes

Boolean attributes use {true} or just the attribute name.

```
<input disabled />
<input checked={true} />
```

```
<input disabled={false} />
<input checked="true" />
```

✖ Don't use strings for booleans.

JSX SYNTAX RULES (2 OF 2)

Four more essential rules, plus a quick-reference table of common JSX patterns.

ADDITIONAL JSX RULES

- **7. Quote string literals —** title="Hello" or title='Hello' -- never title=Hello.
- **8. Key for list items —** {items.map(i => <li key={i.id}>{i.name})} -- a unique, stable key on every mapped element.
- **9. Comments —** use {/* comment */} for multi-line comments inside JSX.
- **10. Render nothing with null —** return null; renders nothing at all -- not an empty string, not undefined.

COMMON EXAMPLES

Purpose	JSX Syntax
Image	
Link	React Docs
Button	<button onClick={handleClick}>Click</button>
Input	<input value={name} onChange={...} />
Conditional	{isLoggedIn ? <Dashboard /> : <Login />}
List Rendering	{items.map(i => <Item key={i.id} {...i} />)}

KEY TAKEAWAY Following JSX syntax rules ensures your components work correctly and your code remains clean and readable -- especially the key rule, which the Component Reusability and Lab 33 slides return to.

TRY IT YOURSELF — EXERCISE 3: JSX ON PAPER

Reinforces: sketching a JSX tree by hand with correct keys — a documentation exercise, no runtime needed.

TARGET SHAPE — SKETCH THIS ON PAPER FIRST

CustomerList (hand-written JSX tree)

```
<CustomerList>
  <CustomerCard key="CUS-1001">
    <h3>Amina Khan</h3>
    <StatusBadge status="ACTIVE" />
  </CustomerCard>
  <CustomerCard key="CUS-1002">
    <h3>Ravi Singh</h3>
    <StatusBadge status="PROSPECT" />
  </CustomerCard>
</CustomerList>
```

STEPS (IN notes/lab33-jsx-tree.md)

#	What To Do
1	Tree: sketch <CustomerList> containing two <CustomerCard> nodes, by hand.
2	Keys: write why key={customerId} should be "CUS-1001", not the array index.
3	Badge: nest <StatusBadge status="ACTIVE" /> inside Amina's card.
4	No Runtime: do not create a Vite app for this exercise -- paper only.

This exact tree — two <CustomerCard> nodes keyed by customerId, each nesting a <StatusBadge> — is what Lab 33's CustomerList.tsx renders from the seed fixtures array.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 3 before continuing: exercises/exercise-03-jsx-paper.md (workspace: examples/module-33-exercises).

RENDERING DYNAMIC CONTENT

React makes it easy to render dynamic data in your UI by embedding JavaScript expressions inside JSX using curly braces {}.

EXAMPLES OF RENDERING DYNAMIC CONTENT

Kind	JSX Pattern
Variable	const name = "Amina"; Hello, {name}!
Expression	const a=10,b=20; Sum is {a + b}
Object Property	{user.name} is a {user.role}
Array (List)	{items.map(i => <li key={i}>{i})}
Conditional	{isActive ? "Active" : "Inactive"}

WAYS TO RENDER DYNAMIC CONTENT

- **Variables**
- **Expressions**
- **Object Properties**
- **Arrays & Lists**
- **Conditional Rendering**

COMPLETE EXAMPLE — CustomerCard.tsx

```
function CustomerCard({ customer }: Props) {
  return (
    <article>
      <h3>{customer.fullName}</h3>
      <p>Status: {customer.status}</p>
      <ul>
        {customer.tags.map((tag) => (
          <li key={tag}>{tag}</li>
        ))}
      </ul>
    </article>
  );
}
```

Keep logic in JavaScript, JSX for presentation; use meaningful keys; handle null/undefined data safely.

KEY TAKEAWAY By embedding expressions inside JSX, React lets you render dynamic data efficiently and keeps your UI in sync with application state and props.

Rendering Dynamic Content

Use JavaScript inside JSX to render dynamic data and expressions.

JavaScript Expressions
Use {} to embed any JavaScript expression.

Variables
Render values stored in variables.

Expressions
Render results of expressions or functions.

Lists
Render lists using .map() and unique keys.

Conditional Rendering
Show different content based on conditions.

JSX Code

```jsx
function UserProfile({ user, isLoggedIn }) {
  const items = ['React', 'JavaScript', 'CSS'];
  const age = user.age;
  const greeting = `Hello, ${user.name}!`;

  return (
    <div className="profile">
      <h2>{greeting}</h2>
      <p>Age: {age}</p>

      <h3>Skills</h3>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>`
        ))}
```

What Happens

1. **Evaluate JavaScript**
   Variables, expressions, and functions are evaluated.
2. **Resolve JSX**
   JSX is converted to React elements.
3. **Reconcile**
   React compares with previous UI and updates what changed.
4. **Update DOM**
   The real DOM is updated to reflect the changes.

Rendered Output (UI)

Hello, Alice!

Age: 28

Skills

- React
- JavaScript
- CSS

Dynamic data and expressions are rendered in the UI.

Common Examples

Variables
`<p>{user.name}</p>`
Renders the value of a variable.

Expressions
`<p>{2 + 3}</p>`
Renders the result of an expression.

Function Calls
`<p>{getStatus()}</p>`
Renders the result of a function.

Lists
`{items.map(item => <li key={item.id}>{item.name}</li>)}`
Renders a list of items.

Conditional Rendering
`{isLoggedIn ? <Dashboard /> : <Login />}`
Renders based on a condition.

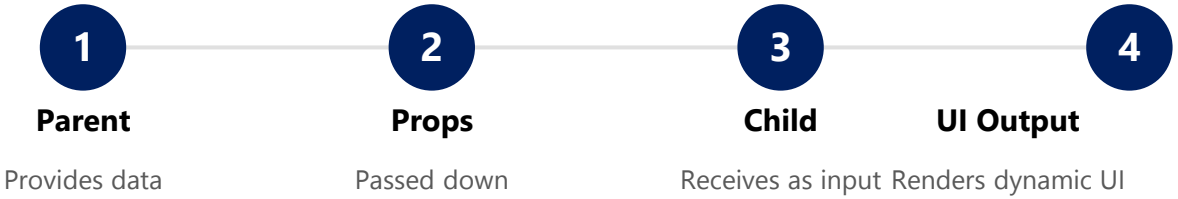
UNDERSTANDING PROPS

Props (short for "properties") are read-only inputs passed from parent to child components to make components dynamic and reusable.

KEY POINTS

- Props flow in one direction: Parent → Child.
- Props are immutable (read-only) inside the child -- a child can never modify its own props.
- Props can carry any data: string, number, boolean, array, object, even functions.
- Access props via the props object, or (recommended) via destructuring.

HOW PROPS WORK



ACCESSING PROPS — CustomerCard.tsx

```
interface Props {
  customer: Customer;
  onEdit: (id: string) => void;
}
// Destructuring (recommended)
function CustomerCard({ customer, onEdit }: Props) { ... }
```

EXAMPLE: PASSING PROPS — App.tsx

```
<CustomerCard
  customer={{ customerId: "CUS-1001", fullName: "Amina Khan",
    status: "ACTIVE" }}
  onEdit={handleEdit}
/>
```

Read-only **Any data type** **Parent → Child**

Destructuring recommended

KEY TAKEAWAY Props are the primary way to pass data from parent to child components -- they are read-only and enable reusable, dynamic, and maintainable components.

Understanding Props

Props (short for properties) are read-only inputs passed from a parent component to a child component.

One-Way Data Flow
Props flow from parent to child.

Read-Only
Child components cannot modify props.

Reusability
Make components dynamic and reusable.

Any Data Type
Props can be strings, numbers, arrays, objects, functions, etc.

Immutable
Props are immutable within the child component.

1. Parent Component (Passes Props)

```jsx
function App() {
  const user = {
    name: 'Alice',
    age: 28,
    skills: ['React', 'JavaScript']
  };

  return <UserProfile user={user} />;
}
```

App (Parent)
Passes the user object to UserProfile as a prop named "user".

2. Child Component (Receives Props)

```jsx
function UserProfile(props) {
  const { user } = props;

  return (
    <div className="profile">
      <h2>{user.name}</h2>
      <p>Age: {user.age}</p>
      <p>Skills: {user.skills.join(', ')}</p>
    </div>
  );
}
```

UserProfile (Child)
Receives props as an argument. Destructures and uses the data to render UI.

3. Rendered Output (UI)

Alice

Age: 28

Skills: React, JavaScript

The child component displays the data passed by the parent via props.

Key Points

Props are Read-Only
Children cannot change props.

Passed by Parent
Data is passed from parent component.

Improve Reusability
Same component can be used with different props.

Use Destructuring
Easily extract props in the child component.

Don't Mutate Props
Treat props as immutable data.

TRY IT YOURSELF — EXERCISE 4: PROPS SKETCH

Reinforces: defining a typed props contract for CustomerCard — an architecture exercise.

CUSTOMERCARD PROPS REFERENCE

Prop	Amina Example	Ravi Example
customerId	CUS-1001	CUS-1002
name	Amina Khan	Ravi Singh
status	ACTIVE	PROSPECT
onSelect	() => void	() => void

STEPS (IN notes/lab33-props.md)

#	What To Do
1	Copy Props: copy the table at left; you've already added the Ravi (CUS-1002) row.
2	Types: write TypeScript-ish types, e.g. status: 'ACTIVE' | 'SUSPENDED' | 'PROSPECT'.
3	Children?: decide, in one sentence, whether CustomerCard takes children or props only.
4	Anti-Pattern: note -- never pass the entire global store as one mega-prop.

Lab 33's real CustomerCard uses exactly this shape: interface Props { customer: Customer; onEdit: (id: string) => void; } -- your sketch previews that same typed contract.

KEY TAKEAWAY TRY IT NOW — Complete Exercise 4 before continuing: exercises/exercise-04-props-sketch.md (workspace: examples/module-33-exercises).

PASSING DATA BETWEEN COMPONENTS (1 OF 2)

React uses one-way data flow to keep your application predictable and easy to manage. Data flows down from parent to child using props.

DATA FLOW PRINCIPLE

1. **Parent Component** — Owns the data
2. **Pass Data (Props)** — Down the tree
3. **Child Component** — Receives as props
4. **Browser UI** — Displays updated UI

1. PASSING DATA DOWN (PARENT TO CHILD)

The parent sends data to the child component using props.

```
// Parent
<CustomerCard customer={amina} onEdit={handleEdit} />

// Child
function CustomerCard({ customer }: Props) {
  return <h3>{customer.fullName}</h3>;
}
```

WHY THIS APPROACH?

- **Makes components reusable —** the same child renders differently for whatever props it's given.
- **Easy to debug and test —** trace a bug by following props down one direction only.
- **Predictable data flow —** you always know where data originates.
- **Better performance —** React can optimize re-renders when data flow is one-directional.

KEY TAKEAWAY Use props to pass data from parent to child, in one predictable direction -- this keeps your app easy to debug, test, and scale.

Passing Data Between Components

In React, data flows down from parent to child via props.

One-Way Data Flow
Parent component passes data to child component using props.

Props are Read-Only
Child component cannot modify props. It can only read them.

Updates Flow Down
When parent state changes, new props are passed to children and UI updates.

PARENT COMPONENT

Manages State
Holds the data (source of truth).

Renders Child
Passes data to child using props.

Passes Data via Props

```
<Child name="Alice"
age={28} skills={["React", "JS"]} />
```

CHILD COMPONENT

Receives Props
Access props using function parameters.

Uses Data
Displays the data in the UI.

Renders UI

Name: Alice
Age: 28
Skills:
- React
- JavaScript

PARENT COMPONENT (App.js)

```
1  import Child from './Child';
2
3  function App() {
4     const user = {
5       name: 'Alice',
6       age: 28,
7       skills: ['React', 'JavaScript"]
8     };
8
9    return <Child name={user.name} age={user.age} skills={user.skills} />;
10 }
```

CHILD COMPONENT (Child.js)

```
1  function Child(props) {
2     const { name, age, skills } = props;
3
4    return (
5      <div className="card">
6        <h2>{name}</h2>
7        <p>Age: {age}</p>
7        <p>Skills: {skills.join(', ')}</p>
8      </div>
9    );
10 }
```

KEY POINTS

- Data flows from Parent to Child.
- Props are immutable (read-only).
- When parent data changes, child re-renders with new props.
- Use props to make components dynamic and reusable.

PASSING DATA BETWEEN COMPONENTS (2 OF 2)

To send data back up, children use callbacks passed down as props. Siblings that share data lift state up to a common parent.

2. PASSING DATA UP (CHILD TO PARENT)

The child sends data back to the parent by calling a callback function the parent passed down as a prop.

```
// Parent owns the data and the handler
function App() {
  const handleEdit = (id: string) =>
    console.log("edit", id, "lab-request-001");
  return <CustomerCard customer={amina} onEdit={handleEdit} />;
}
// Child calls the callback prop -- never mutates data itself
<button onClick={() => onEdit(customer.customerId)}>Edit</button>
```

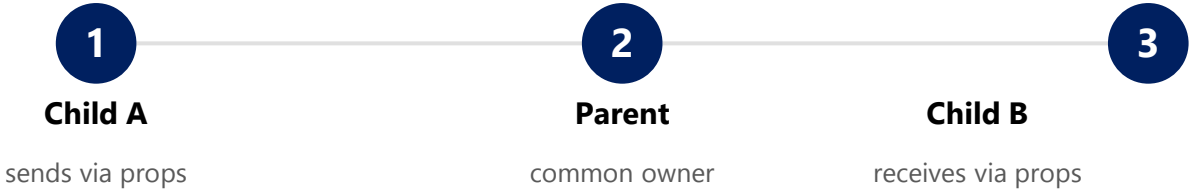
3. SIBLING COMMUNICATION

Siblings cannot talk directly. Lift the shared data up to their common parent, which then passes it down to both as props.

WHY THIS APPROACH?

- **Makes components reusable**
- **Easy to debug and test**
- **Predictable data flow**
- **Better performance**

LIFTING STATE UP



This is exactly what Lab 34 will do to this lab's presentational components: lift the customer array into useState() in App, without rewriting any CustomerCard markup -- because the props CONTRACT (customer, onEdit) stays stable.

KEY TAKEAWAY Use props to pass data down; use callbacks to pass data up; lift state up when siblings need to share data -- one-way data flow keeps your app predictable, maintainable, and easy to scale.

COMPONENT REUSABILITY

Reusability is a core advantage of React. You can build a component once and use it many times with different data and behavior.

WHY REUSE COMPONENTS?

- **Avoid Duplication —** write once, use everywhere.
- **Save Time —** build features faster with existing building blocks.
- **Easy Maintenance —** update in one place, reflected everywhere it's used.
- **Consistency —** the same look and feel across the whole application.
- **Scalability —** easier to manage as the application grows.

EXAMPLES OF REUSABLE COMPONENTS

Category	Examples
UI Components	Button, Card, Input, Modal, StatusBadge
Layout Components	Header, Sidebar, Footer, AppLayout
Form Components	TextInput, Select, Checkbox, CustomerForm
Data Components	Table, List, Pagination, CustomerList
Feedback Components	Alert, Toast, Spinner, EmptyState

TIPS FOR BUILDING REUSABLE COMPONENTS

Small & focused | **Customize via props** | **Sensible defaults** | **Composable** | **Avoid hardcoding**

KEY TAKEAWAY Reusable components make your React applications more efficient, consistent, and maintainable. Build once, use everywhere.

Component Reusability

Build once, use everywhere. Reusable components increase consistency and speed up development.

Reusable
Build a component once and use it in many places.

Consistent
Ensures consistent UI/UX across the application.

Efficient
Saves time and effort by avoiding duplicate code.

Maintainable
Easier to update and maintain in one place.

Scalable
Encourages modular and scalable architecture.

1. Reusable Component (Button.js)

```jsx
function Button({ label, variant = 'primary',
       onClick }) {
  const baseClass = "px-4 py-2 rounded font-medium";
  const variantClass =
    variant === 'primary'
     ? 'bg-blue-600 text-white hover:bg-blue-700'
     : 'bg-gray-200 text-gray-900 hover:bg-gray-300';
  return (
    <button className="${baseClass} ${variantClass}}
      onClick={onClick}>
      {label}
    </button>
  );
}
```

Single Responsbility
This component only handles how a button looks and behaves. It can be reused anywhere.

2. Using the Component in Multiple Places

In Header Component

```jsx
<Button label="Login" variant="primary"
       onClick={handleLogin} />
```

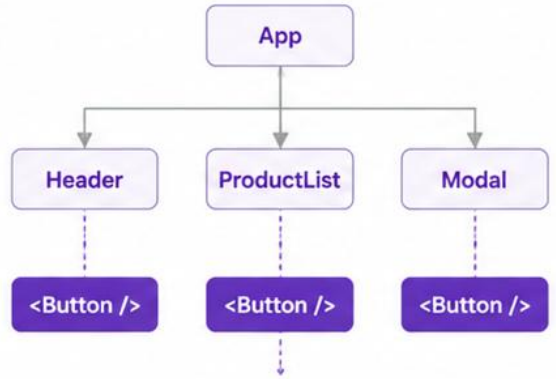
In Product Card Component

```jsx
<Button label="Add to Cart" variant="primary"
       onClick={addToCart} />
```

In Modal Component

```jsx
<Button label="Close" variant="secondary"
       onClick={closeModal} />
```

3. Reusability in Action



One Component, Many Uses
The same Button component is used in different parts of the app with different props.

Key Takeaways

- Components should be small, focused, and reusable.
- Use props to make components flexible and dynamic.
- Reusability leads to faster development.
- Maintain in one place, benefit everywhere.

DESIGNING MODULAR USER INTERFACES (1 OF 2)

Modular UI design means breaking a user interface into small, reusable components that work together to build complex, maintainable applications.

FROM MONOLITH TO MODULAR

1. **Monolithic UI**
   Hard to maintain, repetitive
2. **Modular UI**
   Reusable, testable components
3. **Better Experience**
   Consistent UI, faster delivery

PRINCIPLES OF MODULAR UI DESIGN

- **Single Responsibility —** each component should do one thing and do it well.
- **Reusability —** build components that can be used in multiple places.
- **Composition Over Inheritance —** combine small components to build complex UIs.
- **Consistent Design —** follow a design system for colors, typography, spacing.
- **Separation of Concerns —** keep UI, logic, and data separated.

BENEFITS

Faster Development
Reuse components and speed up building new features.

Easy Maintenance
Update in one place, reflect everywhere.

Scalability
Easily scale the application as requirements grow.

DESIGN SYSTEM INTEGRATION

Shared Colors | Typography | Spacing | Buttons

KEY TAKEAWAY Designing modular user interfaces with reusable components makes your React applications cleaner, easier to maintain, and delightful for users.

Designing Modular User Interfaces

Build UI as a collection of independent, reusable components for scalability, maintainability, and consistency.

Modular
Break UI into small, focused components.

Reusable
Use components across multiple screens.

Consistent
Maintain uniform look and feel with shared styles.

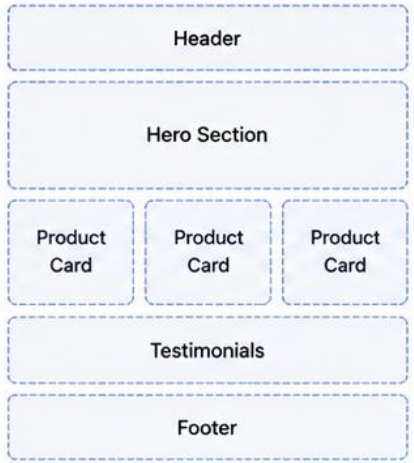
Composable
Combine components to build complex interfaces.

Maintainable
Easier to update, test, and scale over time.

Scalable
Support growth with a flexible component library.

1. Plan: Break Down the UI

Identify sections and break the UI into modular components.



2. Build: Create Reusable Components

Build small, reusable components with single responsibility.

Header
`<Header />`

ProductCard
`<ProductCard />`

Testimonial
`<Testimonial />`

Footer
`<Footer />`

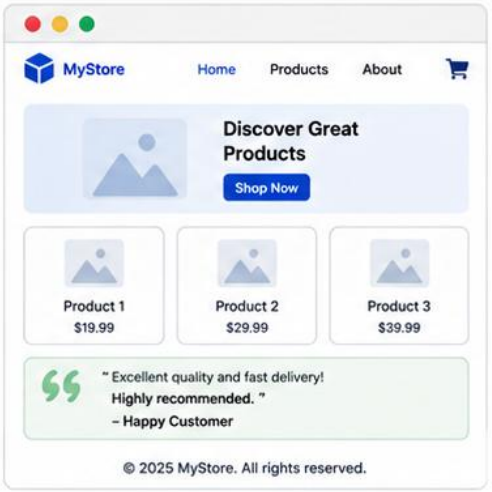
3. Compose: Assemble the UI

Compose the UI by combining components.

```
function HomePage() {
  return (
    <>
      <Header />
      <HeroSection />
      <div className="grid">
        <ProductCard />
        <ProductCard />
        <ProductCard />
      </div>
      <Testimonial />
      <Footer />
    </>
  );
}
```

4. Result: Consistent, Scalable UI

A consistent UI built from reusable components across the application.



Best Practices

- Organize components in a clear folder structure.
- Use shared styles and design tokens for consistency.
- Keep components small and focused on a single task.
- Document components with props and usage examples.
- Test components individually for reliability.

TRY IT YOURSELF — EXERCISE 5: A11Y CHECKLIST

Reinforces: planning accessibility checks before coding the CRM dashboard — a documentation exercise.

ACCESSIBILITY CHECKS TO PLAN (notes/lab33-a11y-checklist.md)

Check	What To Confirm
1 — Semantics	Prefer <button>, <h1>-<h3>, and / over clickable s with onClick handlers.
2 — Contrast	Status colors need visible text or an icon/shape alongside them, never color alone.
3 — Keyboard	Confirm Tab order can reach the View action for both Amina's and Ravi's cards.
4 — Checklist File	Save five checkbox lines covering these checks in notes/lab33-a11y-checklist.md.

WHAT GOOD A11Y LOOKS LIKE IN JSX

CustomerCard.tsx (accessible shape)

```
<article aria-label={`${name} (${customerId})`}>
  <h3 id={`customer-${customerId}`}>{name}</h3>
  <StatusBadge status={status} />  {/* renders TEXT, not color alone */}
  <button type="button" onClick={() => onSelect(customerId)}>
    View
  </button>
</article>
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 5 before continuing: exercises/exercise-05-a11y-checklist.md (workspace: examples/module-33-exercises).

DESIGNING MODULAR USER INTERFACES (2 OF 2)

A worked example: breaking a dashboard UI into a component hierarchy, then composing it back together in code.

1. BREAK DOWN THE UI INTO A HIERARCHY

- App
  - AppLayout (Header, main landmark)
  - CustomerToolbar (Add button)
  - CustomerList
    - CustomerCard (repeated, keyed by customerId)
      - StatusBadge
  - CustomerForm

2. COMPONENT COMMUNICATION

- **Parent → Child —** App passes customers down to CustomerList via props.
- **Child → Parent —** CustomerCard sends edit intent up via the onEdit callback.

3. EXAMPLE CODE (COMPOSITION) — App.tsx

```
function App() {
  return (
    <AppLayout>
      <CustomerToolbar onAdd={handleAdd} />
      <CustomerList
        customers={seedCustomers}
        onEdit={handleEdit}
      />
      <CustomerForm value={draft} errors={{}}
        onChange={handleChange} onSubmit={handleSubmit}
        onCancel={handleCancel} />
    </AppLayout>
  );
}
```

BEST PRACTICES

Start small | **Meaningful names** | **Extract repeated UI** | **Document components**

KEY TAKEAWAY Break the UI into components, build each reusable piece, compose them together, and deliver a great, consistent user experience -- this is exactly Lab 33's build order.

REACT BEST PRACTICES (1 OF 2)

Following best practices helps you build React applications that are maintainable, scalable, performant, and easy to work with.

#	Practice	What It Means
1	Use Functional Components	Prefer functional components with Hooks over class components.
2	Use Hooks Effectively	Use state and effect Hooks appropriately; avoid unnecessary effects.
3	Keep Components Small and Focused	Each component should do one thing well (Single Responsibility).
4	Use Props for Communication	Pass data via props; keep components pure and predictable.
5	Optimize Performance	Use React.memo, useMemo, useCallback to prevent unnecessary re-renders.
6	Organize Your Code	Follow a clear folder structure and consistent naming conventions.
7	Handle Errors Gracefully	Use error boundaries and proper fallback UI.
8	Write Clean and Consistent Code	Use ESLint, Prettier, and a consistent code style.

CORE PRINCIPLES TO FOLLOW

Keep It Simple **Reusability** **Performance** **Reliability** **Consistency**

KEY TAKEAWAY Good practices today lead to better code, faster development, and fewer bugs tomorrow -- easier to maintain, better performance, fewer bugs and issues.

React Best Practices

Follow these best practices to build maintainable, performant and scalable React applications.

1. Component Composition

- Prefer composition over inheritance.
- Build small, reusable components.

```
<Card>
  <Card.Header />
  <Card.Body />
  <Card.Footer />
</Card>
```

2. Reusable Components

- Create reusable components with single responsibility.
- Use props to make them flexible.

```
function Button({ label, onClick }) {
  return (
    <button onClick={onClick}>
      {label}
    </button>
  );
}
```

3. Props & State Management

- Use props for passing data down.
- Keep state as local as possible.

```
function User({ name }) {
  return <h2>{name}</h2>;
}
```

4. Hooks Best Practices

- Use hooks at the top level only.
- Extract custom hooks for reusability.

```
function useFetch(url) {
  // custom hook logic
}

const data = useFetch('/api/data');
```

5. File & Folder Structure

- Organize files by feature, not by type.
- Keep structure scalable and clean.

```
src/
├── features/
│   └── user/
│       ├── components/
│       ├── hooks/
│       ├── services/
│       └── UserPage.jsx
├── shared/
├── App.jsx
```

6. Performance Optimization

- Use React.memo, useMemo and useCallback where needed.
- Avoid unnecessary re-renders.

```
const Memoized = React.memo(MyComp);
const memoValue = useMemo(() =>
 computeExpensive(value), [value]);
```

7. Lists & Keys

- Use key prop for list items.
- Keys should be stable and unique.

```
{items.map(item => (
  <Item key={item.id} data={item} />
))}
```

8. Conditional Rendering

- Render UI based on conditions.
- Keep components clean and readable.

```
return isLoggedIn ? (
  <Dashboard />
) : (
  <Login />
);
```

9. Code Quality & Consistency

- Use ESLint and Prettier.
- Follow consistent naming conventions.
- Write self-documenting code.

```
// Good
const isLoading = true;
function handleSubmit() { ... }
```

10. Testing

- Write unit tests for components and hooks.
- Aim for high coverage of critical parts.

```
test('renders button', () => {
  render(<Button label="Click" />);
  expect(screen.getByText('Click'))
    .toBeInTheDocument();
});
```

Key Takeaways

- Build small, focused components.
- Promote reusability and composition.
- Optimize performance and avoid re-renders.
- Maintain clean code and structure.
- Test your components for reliability.

REACT BEST PRACTICES (2 OF 2)

Best practices in code: a functional component with a Hook, a reusable component with props, and a recommended folder structure.

FUNCTIONAL + A HOOK — Counter.jsx

```
import { useState } from "react";
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

REUSABLE WITH PROPS — Button.jsx

```
function Button({ label, variant = "primary", onClick }) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {label}
    </button>
  );
}
// <Button label="Save" variant="primary" onClick={save} />
```

RECOMMENDED COMPONENT STRUCTURE

- **components/ —** reusable UI components (Button, StatusBadge, CustomerCard).
- **pages/ —** page-level components (Dashboard, Profile).
- **hooks/ —** custom React Hooks (useFetch).
- **utils/ —** utility functions and API calls.
- **App.jsx / index.jsx —** root component and entry file.

ADDITIONAL TIPS

Meaningful names **Avoid deep nesting** **Controlled forms**

Keys on lists **Test components**

KEY TAKEAWAY Following best practices helps you build React applications that are clean, efficient, scalable, and delightful for users and developers alike.

TRY IT YOURSELF — EXERCISE 6: LAB 33 READINESS CHECKLIST

Capstone pre-lab check — confirm component contracts and workspace before opening Lab 33.

READINESS CHECKLIST (RECORD IN notes/lab33-readiness.md)

Check	What To Confirm
Folder Plan	Note the future path: crm-ui/src/components/ (created in the lab, not now).
Evidence	Plan a future screenshot: the list view showing Amina + Ravi cards (lab, not now).
Boundary	Confirm no npm install or npm start is required for this pre-lab.
Pass Mark	Pass if your Exercise 4 (props) and Exercise 1 (TODOs) notes both exist.

Setup (PowerShell) — from EXERCISES-INDEX.md

```
cd $env:USERPROFILE\java-bootcamp
New-Item -ItemType Directory -Force -Path examples\module-33-exercises | Out-Null
cd examples\module-33-exercises
java -version
```

KEY TAKEAWAY TRY IT NOW — Complete Exercise 6 before opening Lab 33: exercises/exercise-06-lab33-readiness.md.

LAB OVERVIEW -- REACT COMPONENTS FOR THE CRM DASHBOARD

Lab 33: React Components for the CRM Dashboard -- the Customer Management Platform React client.

BUSINESS SCENARIO

- Before state (Lab 34) and API integration (Lab 35), leadership freezes the UI contract: every customer surface must be typed, composable, and accessible -- color-only status, index keys, and class-name-only tests are all rejected.
- You own the gate for Amina (ACTIVE) and Ravi (PROSPECT) cards, empty-list UX, and an accessible form shell.
- This lab builds the presentational shell only -- fixtures, no API yet -- so Lab 34 can lift state without rewriting markup.

LEARNING OBJECTIVES

- Scaffold a React TypeScript CRM app with Vite.
- Define typed Customer, CustomerStatus, and CustomerDraft models.
- Create accessible StatusBadge and CustomerCard components.
- Compose CustomerList from cards with stable customerId keys.
- Build a labeled CustomerForm and empty/loading/error shells.
- Write React Testing Library tests that query by role, not class name.

WHAT YOU BUILD

- Vite React-TS crm-ui under lab33-crm; typed models + seed fixtures Amina/Ravi; StatusBadge, CustomerCard, CustomerList, CustomerForm, layout shells; empty/loading/error presentation components; RTL tests; npm run build success.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; correlation id lab-request-001 appears on every logged edit/add callback.

LAB TASKS AND DELIVERABLES

React Components for the CRM Dashboard -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Scaffold Vite React-TS crm-ui; install Vitest + React Testing Library.
2	Define Customer / CustomerStatus / CustomerDraft types + seed fixtures Amina/Ravi.
3	Create StatusBadge -- visible text status, not color alone.
4	Create CustomerCard -- <article>, heading id, mailto link, Edit button.
5	Compose CustomerList with stable customerId keys + EmptyState.
6	Create labeled CustomerForm with htmlFor/id pairs + role="alert" errors.
7	Compose the dashboard: AppLayout, CustomerToolbar, one main landmark.
8	Add LoadingState / ErrorState shells; write RTL tests by role; evidence pack.

EXPECTED DELIVERABLES

- Complete crm-ui project, organized folder structure.
- Reusable components with props and clean code.
- Parent-to-child and child-to-parent data flow.
- RTL tests green; npm run build succeeds.
- README runbook + component notes + screenshots.

RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/A11y 10 · Docs 10

KEY TAKEAWAY Class-name-only tests lose automated-verification marks; index keys are an honor violation; color-only status without visible text is an accessibility failure.

MODULE 33 SUMMARY

In this module, we learned how to build modular, reusable, and maintainable user interfaces in React by working with components, props, JSX, and best practices.

KEY CONCEPTS COVERED

JSX Syntax

Write HTML-like code in JavaScript using JSX, transpiled by Babel.

Functional Components

Build components as plain functions that return JSX.

Understanding Props

Pass typed, read-only data from parent to child components.

Passing Data

Data down via props, events up via callbacks -- one-way data flow.

Composition & Reuse

Combine small, reusable, well-documented components into complex UIs.

Best Practices

Modular design, accessibility, and clean, consistent code organization.

MODULE FLOW RECAP

1. **JSX & Components**
2. **Props & Data Flow**
3. **Composition & Reuse**
4. **Modular UI Design**
5. **Lab: CRM Dashboard**

KEY TAKEAWAY React components are the building blocks of modern user interfaces. By mastering JSX, props, composition, and best practices, you can build powerful, reusable, and scalable applications.

KNOWLEDGE CHECK (1 OF 3)

Test your understanding of React components, props, state, and JSX -- questions 1-4 of 10.

1. What is the primary purpose of a React component?

A. To store data in the browser

B. To define reusable pieces of UI

C. To handle routing in the application

D. To connect to the database

2. Which of the following is used to pass data from a parent to a child component?

A. useState

B. useEffect

C. Props

D. useContext

3. How do you access a prop named "title" in a functional component?

A. props.title

B. this.props.title

C. title.props

D. state.title

4. Which Hook is used to manage local state in a functional component?

A. useEffect

B. useState

C. useReducer

D. useContext

KEY TAKEAWAY A component defines a reusable piece of UI; props pass data down; access a prop via props.title (functional components have no this); useState manages local state.

KNOWLEDGE CHECK (2 OF 3)

Four more questions on re-renders, reusability, callbacks, and modular design -- questions 5-8 of 10.

5. What will trigger a re-render in a functional component?

A. Updating state using useState
B. Changing a prop value
C. Calling a regular JavaScript function
D. All of the above

6. Which of the following helps in making components reusable?

A. Hardcoding values inside the component
B. Using props to accept dynamic data
C. Duplicating the component code
D. Using global variables

7. What is the correct way to pass a function from a parent to a child component?

A. By using props
B. By using state
C. By using context
D. By using ref

8. Which principle suggests breaking the UI into small, independent pieces?

A. Separation of Concerns
B. Component Reusability
C. Modular UI Design
D. All of the above

KEY TAKEAWAY State changes, prop changes, and calling a regular function can all trigger a re-render; props (not global variables) make components reusable and pass callbacks down.

KNOWLEDGE CHECK (3 OF 3)

The final two questions, on maintainability and file conventions -- questions 9-10 of 10.

9. Which practice helps keep components easy to maintain?

A. Writing large components with multiple responsibilities

B. Using meaningful names and keeping components small

C. Avoiding props to reduce complexity

D. Using inline styles everywhere

10. Which file extension is commonly used for React components?

A. .js

B. .jsx

C. .ts

D. .html

KEY TAKEAWAY Meaningful names and small, focused components stay maintainable; .jsx (and .tsx for typed components, as in Lab 33) is the conventional React component file extension.

"Build once. Compose everywhere. Ship reusable UI."

MODULE 33 COMPLETE

React Component Architecture

You can now build functional React components, write JSX, pass data with typed props, compose reusable pieces, and design modular, accessible user interfaces.